🚀 Guía Completa de Instalación y Condiciones de Ejecución — Sistema Multiagente Urbanístico (NITEC)

Este documento reúne **todos los pasos de instalación desde cero** para configurar el sistema en un ordenador nuevo, así como las **condiciones obligatorias (Checklist Pre-vuelo)** que deben cumplirse para garantizar su funcionamiento.

---

1. Requisitos del Sistema (Hardware y SO)

- **Sistema Operativo:** Windows 10 / 11 (64 bits), macOS o Linux.
- **Memoria RAM:** Mínimo 8 GB (Recomendado 16 GB o superior para ejecutar `llama3.1` con contexto amplio de 16000 tokens).
- **Almacenamiento Libre:** Al menos 10 GB libres en disco (para Ollama, los modelos de IA y las bases de datos vectoriales ChromaDB).
- **Tarjeta Gráfica (GPU):** Opcional pero muy recomendada (NVIDIA con soporte CUDA o Apple Silicon) para acelerar la extracción del Agente 1 y Agente 2. Si no hay GPU dedicada, Ollama usará la CPU automáticamente.

---

2. Guía de Instalación desde Cero (Ordenador Nuevo)

Paso 2.1. Instalar Python (Versión recomendada: Python 3.11 o 3.12)

Se recomienda usar **Python 3.11** o **Python 3.12**, ya que `chromadb` dispone de paquetes precompilados (*wheels*) para estas versiones en Windows y no requiere instalar compiladores de C++ adicionales.

Opción A: Usando Miniconda (Recomendado, igual que tu entorno actual)

1. Descarga el instalador de **Miniconda** desde: https://docs.anaconda.com/miniconda/
2. Ejecuta el instalador con las opciones por defecto.
3. Abre **Anaconda PowerShell Prompt** (o tu terminal de Windows) y verifica la instalación:

```powershell
python --version
```

Opción B: Usando el instalador oficial de Python

1. Descarga Python 3.11 o 3.12 desde: https://www.python.org/downloads/
2. **MUY IMPORTANTE:** En la primera pantalla del instalador en Windows, marca la casilla **"Add python.exe to PATH"** antes de pulsar en *Install Now*.

Paso 2.2. Instalar Ollama y Descargar los Modelos de IA

El sistema utiliza **Ollama** como motor local de Inteligencia Artificial tanto para razonamiento (`llama3.1`) como para la vectorización de documentos (`nomic-embed-text`).

1. **Descargar e instalar Ollama:**
   - Entra en https://ollama.com/download, descarga el instalador para tu sistema operativo y ejecútalo.
2. **Verificar que Ollama está instalado:** Abre una terminal (PowerShell o CMD) y ejecuta:

```powershell
ollama --version
```

3. **Descargar los dos modelos obligatorios del sistema:** En la misma terminal, ejecuta uno por uno los siguientes comandos y espera a que lleguen al `$100\%$`:

```powershell
ollama pull llama3.1
```

```powershell
ollama pull nomic-embed-text
```

4. **Comprobar que ambos modelos están listos:**

```powershell
ollama list
```

(Deben aparecer en la lista tanto `llama3.1:latest` como `nomic-embed-text:latest`).

Paso 2.3. Preparar la Carpeta del Proyecto y Entorno Virtual (Opcional pero recomendado)

1. Crea una carpeta para el proyecto (por ejemplo, `C:\NITEC_SOLUTIONS\Archivos_NITEC_Reales`) y descomprime dentro todos los archivos `.py` del sistema:
    - `main.py`
    - `agente_1_extractor.py`
    - `agente_2_normativo_local.py`
    - `agente_3_procesador.py`
    - `creador_base_datos.py`
    - `creador_base_vectores.py`
    - `geo_localizador.py`
2. Abre la terminal en esa carpeta:

    ```powershell
    cd "Ruta\A\Tu\Carpeta\Archivos_NITEC_Reales"
    ```
3. *(Opcional)* Si quieres crear un entorno limpio con Conda:

    ```powershell
    conda create -n nitec python=3.11 -y
    conda activate nitec
    ```

Paso 2.4. Instalar las Bibliotecas de Python Necesarias

Ejecuta el siguiente comando único en tu terminal para instalar todas las dependencias que utilizan los scripts:

```powershell
pip install langchain langchain-core langchain-community
langchain-text-splitters langchain-ollama langchain-chroma chromadb
pypdf rank-bm25
```

Detalle de por qué es necesaria cada biblioteca:

- `langchain`, `langchain-core`, `langchain-community`, `langchain-text-splitters`: Orquestación de prompts, cadenas y fragmentación de texto (`RecursiveCharacterTextSplitter`).

- `langchain-ollama`: Conexión directa con `ChatOllama` (`llama3.1`) y `OllamaEmbeddings` (`nomic-embed-text`).
- `langchain-chroma` y `chromadb`: Motor de base de datos vectorial persistente en disco (`./bd_vectores` y `./bd_normativas`).
- `pypdf`: Lectura de páginas PDF (`PyPDFLoader`) y corte/división de archivos en el Agente 3 (`PdfReader`, `PdfWriter`).
- `rank-bm25`: **Imprescindible** para que funcione el buscador híbrido por palabras clave (`BM25Retriever`) dentro de `agente_1_extractor.py`.

(Si prefieres usar un archivo `requirements.txt`, puedes crear un archivo con ese nombre, pegar la lista de paquetes uno por línea y ejecutar `pip install -r requirements.txt`).

---

3. Condiciones Obligatorias para Ejecutar el Sistema (Checklist Pre-vuelo)

Antes de ejecutar `python main.py`, asegúrate de que se cumplen estas **6 condiciones**:

✅ Condición 1: El servicio de Ollama debe estar activo en segundo plano

- En Windows, asegúrate de que el icono de la llama de **Ollama** aparece en la bandeja del sistema (abajo a la derecha, junto al reloj), o ejecuta `ollama serve` en una ventana aparte si lo cerraste.
- Si Ollama está apagado, Python lanzará un error `ConnectionRefusedError` al intentar vectorizar o consultar al LLM.

✅ Condición 2: Los modelos `llama3.1` y `nomic-embed-text` deben estar descargados

- El código llama explícitamente a `model="llama3.1"` y `model="nomic-embed-text"`. Si falta alguno de los dos en `ollama list`, el pre-vuelo o los agentes fallarán.

✅ Condición 3: Los PDFs (Proyecto y BOJA) deben tener texto digital seleccionable

- El sistema utiliza `PyPDFLoader` (`pypdf`), que extrae texto digital.
- Si un PDF es una **imagen escaneada pura** (fotocopia sin capa de texto OCR), `PyPDFLoader` extraerá páginas en blanco y el sistema indicará: `"No se pudo extraer texto"`.
- **Comprobación rápida:** Abre el PDF y prueba a seleccionar una frase con el ratón. Si puedes seleccionarla, es válido.

✅ Condición 4: No tener un `boja_base.json` corrupto o incompleto de pruebas antiguas

- En la **primera ejecución**, asegúrate de que **NO** exista ningún archivo `boja_base.json` viejo o vacío en la carpeta del proyecto.
- Así, cuando ejecutes `main.py`, el **Agente 2** leerá el PDF oficial del BOJA y generará el archivo `boja_base.json` limpio y completo de forma automática. Una vez creado bien la primera vez, **se quedará guardado para siempre**.

✅ Condición 5: No tener terminales antiguas de Python colgadas en segundo plano

- Aunque el código nuevo ya libera automáticamente el archivo `chroma.sqlite3` tras cada consulta (`gc.collect()`), evita tener dos consolas ejecutando `main.py` al mismo tiempo sobre la misma carpeta para que Windows no bloquee `./bd_vectores`.

✅ Condición 6: Rutas válidas en Windows

- Puedes pegar las rutas con o sin comillas (ej. `E:\Universidad\...\Proyecto_de_prueba` o `"E:\Universidad\...\Proyecto_de_prueba"`), ya que `main.py` limpia las comillas automáticamente.
- La carpeta del proyecto debe contener al menos un archivo con extensión `.pdf`.

---

4. Estructura de Archivos y Persistencia del Sistema

Cuando el sistema esté funcionando, tu carpeta de trabajo tendrá la siguiente estructura:

```
None
📁 Archivos_NITEC_Reales/
│
├── 🚀 main.py                          # Orquestador principal (Único archivo que debes ejecutar)
├── 🧠 agente_1_extractor.py            # Extrae fase de tramitación y fecha de aprobación
├── ⚖️ agente_2_normativo_local.py      # Genera boja_base.json y guarda límites de Ayuntamientos
├── ✂️ agente_3_procesador.py           # Evalúa el peso en MB y divide los PDFs
├── 🗄️ creador_base_datos.py            # Ingestor vectorial para normativas (./bd_normativas)
```

```
├── 🔄 creador_base_vectores.py       # Ingestor vectorial temporal del proyecto (./bd_vectores)
├── 📍 geo_localizador.py             # Detecta provincia y comunidad autónoma del municipio
│
├── 📄 municipios_espana.json         # (Auto-generado) Base de datos geográfica local
├── 📄 boja_base.json                 # (Auto-generado en 1ª vez) Reglas extraídas del PDF del BOJA
├── 📄 normativas_ayuntamientos.json  # (Auto-generado) Guarda para siempre los MB de cada Ayuntamiento
│
├── 📁 normativas_pdf/                # (Auto-creada) Guarda copia de los PDFs normativos/BOJA
├── 📁 bd_normativas/                 # (Auto-creada) Memoria vectorial permanente de leyes
└── 📁 bd_vectores/                   # (Auto-creada) Memoria vectorial del proyecto en curso
```

---

5. Cómo Ejecutar el Sistema Paso a Paso

1. Abre tu terminal en la carpeta del proyecto y ejecuta:

   ```powershell
   python main.py
   ```

2. **Introduce los datos que te pide por pantalla:**
   - **Municipio:** (ej. `Marbella` o `Málaga`). Detectará la provincia y comunidad automáticamente.
   - **Tipo de proyecto:** (ej. `ED`, `PGOM`, `POU`).
   - **Ruta a la CARPETA del proyecto:** Pega la ruta donde están los PDFs del proyecto urbanístico.
   - **Ruta al PDF principal:** Pega la ruta al PDF principal o simplemente **pulsa `Enter`** para que autodetecte el primer PDF de la carpeta.
3. **Solo en la 1ª ejecución (Configuración Inicial):**
   - Si aún no existe `boja_base.json`, te pedirá la ruta al PDF del BOJA. El **Agente 2** lo analizará y creará `boja_base.json` para siempre.
   - Si es la primera vez que trabajas con ese Ayuntamiento (ej. `Marbella`), te pedirá que introduzcas **a mano** el límite de peso en MB (ej. `15`). El **Agente 2** lo guardará en `normativas_ayuntamientos.json` para no volver a preguntártelo en el futuro.
4. **Resultado:**

- Verás en pantalla el JSON consolidado con los datos geográficos, los metadatos extraídos de la carátula por el Agente 1 y las reglas de empaquetado e identidad consolidadas por el Agente 2.

---

6. Solución Rápida de Problemas (Troubleshooting)

Síntoma / Error en Consola	**Causa**	**Solución**
`ModuleNotFoundError: No module named 'rank_bm25'`	Falta la librería del buscador léxico del Agente 1.	Ejecuta `pip install rank-bm25`
`ConnectionError` o fallo al conectar con `localhost:11434`	Ollama no se está ejecutando o falta descargar un modelo.	Abre la aplicación **Ollama** y ejecuta `ollama pull llama3.1` y `ollama pull nomic-embed-text`
`"El proyecto 'XX' no existe en el BOJA"`	Quedó un `boja_base.json` antiguo de versiones previas.	Borra el archivo `boja_base.json` y vuelve a ejecutar `python main.py`
Quieres cambiar el límite en MB de un Ayuntamiento ya guardado	El límite quedó registrado en `normativas_ayuntamientos.json`.	Abre `normativas_ayuntamientos.json` con cualquier editor de texto y cambia el número, o borra ese archivo para que te lo vuelva a preguntar por consola.